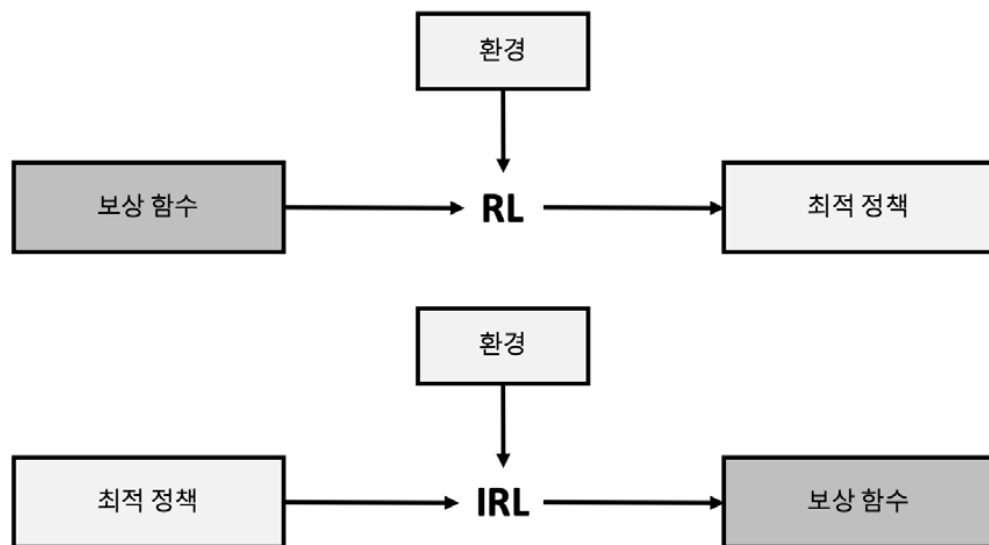


Introduction to Inverse Reinforcement Learning

고려대학교 산업경영공학과
데이터 애널리틱스 및 헬스케어 시스템 연구실(DAHS)
석사과정
김홍범

1. Introduction

1. What is Inverse Reinforcement Learning?

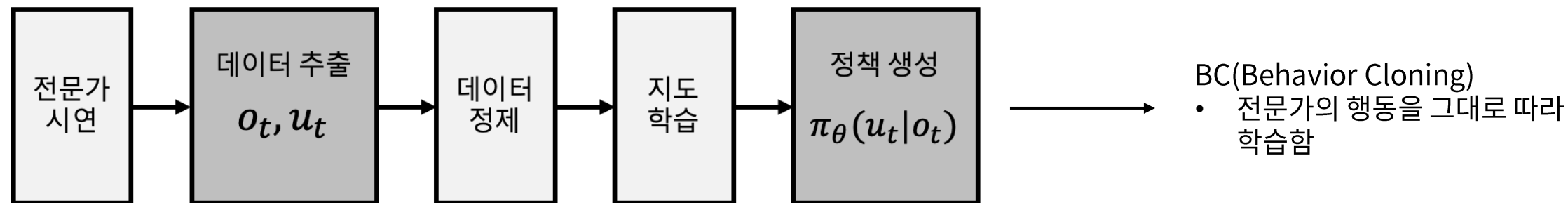


Type	RL	IL
Learn	Learn through rewards	Learn through demonstrations
Policy	Trial and error	No rewards necessary
Speed	Super speed simulation	Real time interaction
Style	Agent becomes "Optimal" at task	Agent becomes "Human - like" at task

1. Introduction

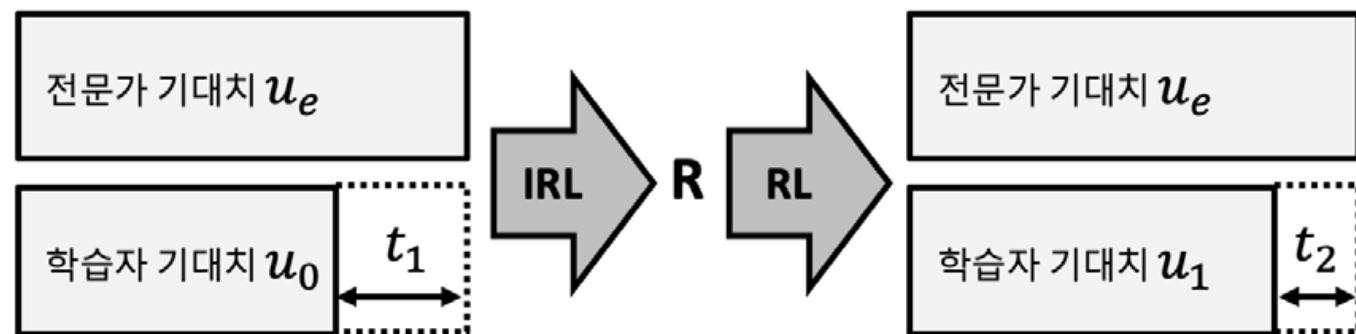
1. What is Inverse Reinforcement Learning?

IL (Imitation Learning)



Dis - Advantage : Overfitting, Sufficient Dataset

IRL(Inverse Reinforcement Learning) = AL(Apprenticeship Learning)



2. Algorithms

1. ALIRL(Apprenticeship Learning via IRL)



Apprenticeship Learning via Inverse Reinforcement Learning

Pieter Abbeel
Andrew Y. Ng

PABBEEL@CS.STANFORD.EDU
ANG@CS.STANFORD.EDU

Computer Science Department, Stanford University, Stanford, CA 94305, USA

Abstract

We consider learning in a Markov decision process where we are not explicitly given a reward function, but where instead we can observe an expert demonstrating the task that we want to learn to perform. This setting is useful in applications (such as the task of driving) where it may be difficult to write down an explicit reward function specifying exactly how different desiderata should be traded off. We think of the expert as trying to maximize a reward function that is expressible as a linear combination of known features, and give an algorithm for learning the task demonstrated by the expert. Our algorithm is based on using “inverse reinforcement learning” to try to recover the unknown reward function. We show that our algorithm terminates in a small number of iterations, and that even though we may never recover the expert’s reward function, the policy output by the algorithm will attain performance close to that of the expert, where here performance is measured with respect to the expert’s *unknown* reward function.

1. Introduction

Given a sequential decision making problem posed in the Markov decision process (MDP) formalism, a number of standard algorithms exist for finding an optimal or near-optimal policy. In the MDP setting, we typically assume that a reward function is given. Given a reward function and the MDPs state transition probabilities, the value function and optimal policy are exactly determined.

The MDP formalism is useful for many problems because it is often easier to specify the reward function than to directly specify the value function (and/or optimal policy). However, we believe that even the reward function is frequently difficult to specify manually. Consider, for example, the task of highway driving. When driving, we typically trade off many dif-

ferent desiderata, such as maintaining safe following distance, keeping away from the curb, staying far from any pedestrians, maintaining a reasonable speed, perhaps a slight preference for driving in the middle lane, not changing lanes too often, and so on To specify a reward function for the driving task, we would have to assign a set of weights stating exactly how we would like to trade off these different factors. Despite being able to drive competently, the authors do not believe they can confidently specify a specific reward function for the task of “driving well.”¹

In practice, this means that the reward function is often manually tweaked (cf. reward shaping, Ng et al., 1999) until the desired behavior is obtained. From conversations with engineers in industry and our own experience in applying reinforcement learning algorithms to several robots, we believe that, for many problems, the difficulty of manually specifying a reward function represents a significant barrier to the broader applicability of reinforcement learning and optimal control algorithms.

When teaching a young adult to drive, rather than telling them what the reward function is, it is much easier and more natural to demonstrate driving to them, and have them learn from the demonstration. The task of learning from an expert is called *apprenticeship learning* (also learning by watching, imitation learning, or learning from demonstration).

A number of approaches have been proposed for apprenticeship learning in various applications. Most of these methods try to directly mimic the demonstrator by applying a supervised learning algorithm to learn a direct mapping from the states to the actions. This literature is too wide to survey here, but some examples include Sammut et al. (1992); Kuniyoshi et al. (1994); Demirir & Hayes (1994); Amit & Mataric (2002); Pomerleau (1989). One notable exception is given in Atkeson & Schaal (1997). They considered the

¹We note that this is true even though the reward function may often be easy to state in English. For instance, the “true” reward function that we are trying to maximize when driving is, perhaps, our “personal happiness.” The practical problem however is how to model this (i.e., our happiness) explicitly as a function of the problems’ states, so that a reinforcement learning algorithm can be applied.

Appearing in *Proceedings of the 21st International Conference on Machine Learning*, Banff, Canada, 2004. Copyright 2004 by the authors.

ICML '04: Proceedings of the twenty-first international conference on Machine learning, July

Pieter Abbeel, Andrew Y. Ng

Citation : 3890

2. Algorithms

1. ALIRL(Apprenticeship Learning via IRL)

Objective : Guess the reward function by using linear combination

finite-state MDP (S, A, T, γ , D) D : initial-state distribution

1. Randomly pick some policy π^0 (via Monte Carlo), compute $\mu^0 = \mu(\pi^0)$

2. Estimate feature expectation set

Feature expectation $E_{s_0 \sim D}[V^\pi(s_0)] = E[\sum_{t=0}^{\infty} \gamma^t \boxed{R(s_t)} | \pi] \quad (1)$

$$= E[\sum_{t=0}^{\infty} \gamma^t \boxed{w \cdot \phi(s_t)} | \pi] \quad (2)$$

$$= w \cdot E[\sum_{t=0}^{\infty} \gamma^t \phi(s_t) | \pi] \quad (3)$$

ϕ : some vector of features

Unknown reward function
 $R^* = w^{*T} \phi$

Expected discounted accumulated feature expectation : $\mu(\pi) = w^* E[\sum_{t=0}^{\infty} \gamma^t \phi(s_t) | \pi] \in \mathbb{R}^k$

—————→ Value of policy(π)

2. Algorithms

1. ALIRL(Apprenticeship Learning via IRL)

3. Compute difference expert expectation and apprenticeship expectation

Expert expectation $\hat{\mu}_E = \frac{1}{m} \sum_{i=1}^m \sum_{t=0}^{\infty} \gamma^t \phi(s_t^{(i)}).$

$$t^{(i)} = \mu^i(\pi) - \hat{\mu}_E$$

$$\max_{t,w} t$$

$$\text{s.t. } w^T \mu_E \geq w^T \mu^{(j)} + t, j = 0, \dots, i-1$$

$$\|w\|_2 \leq 1$$

—————> Linear combination

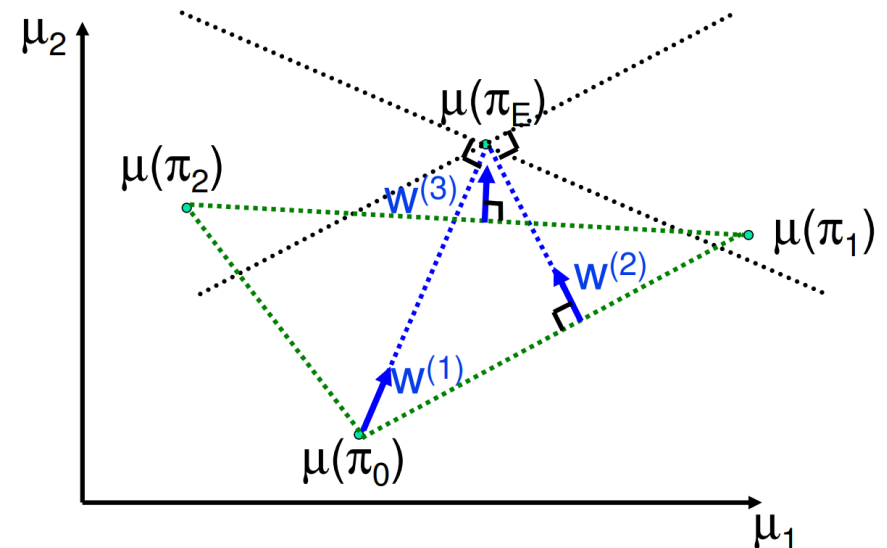
4. $t^{(i)} \leq \varepsilon$, then terminate.

5. Using the RL algorithm, compute the optimal policy π^i for the MDP using $R = (w^{(i)})^T \phi$

6. Compute $\mu^i = \mu(\pi^i)$

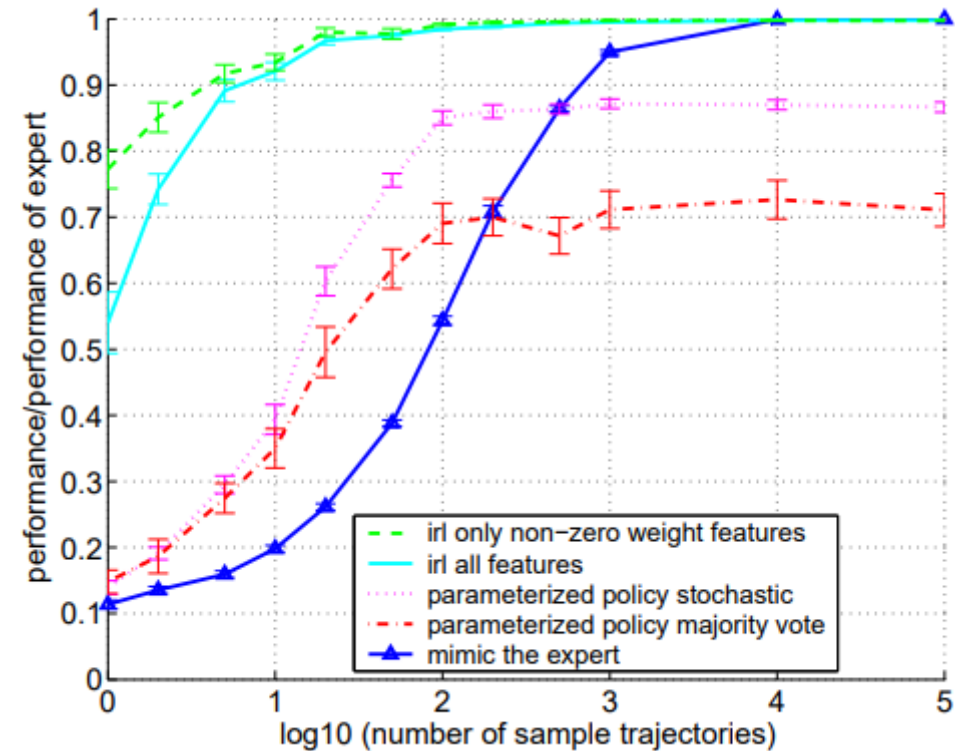
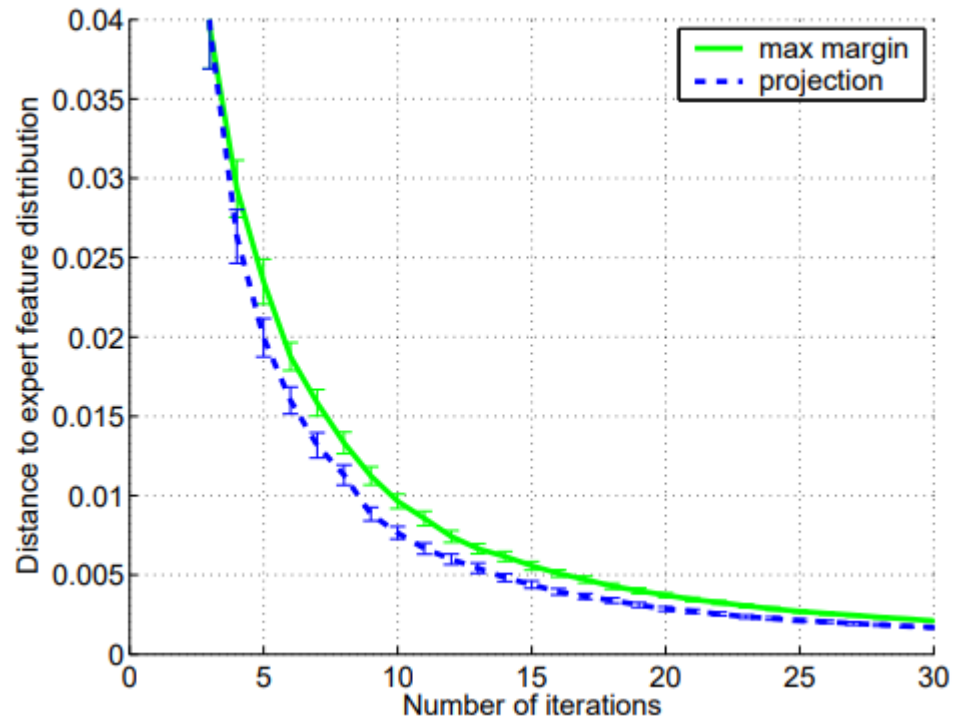
7. Set $i = i + 1$, and go back to step 3

They solve this algorithm QP (or SVM) solver.



2. Algorithms

1. ALIRL(Apprenticeship Learning via IRL)



Grid world Example

2. Algorithms

2. MaxENT IRL(Maximum Entropy IRL)

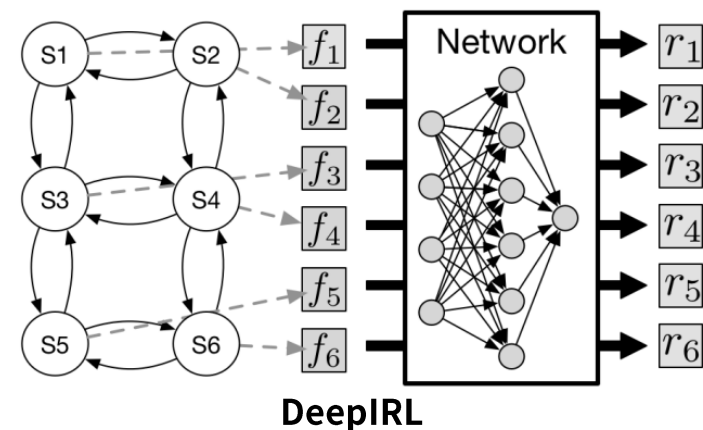
ALIRL : 수많은 정책이 동일한 Feature expectation를 발생시킬 수 있다는 단점을 가짐(각 정책이 수많은 보상 함수에 대해 최적일 수 있음)

MaxENT IRL(Maximum Entropy IRL)

- MaxENT은 관측된 데이터로부터 feature expectation을 따르는 확률 분포를 구함
- 분포의 엔트로피를 최대화

1. 파라미터 초기화
2. 보상에 관한 최적의 정책 계산
3. 상태 방문 빈도 계산
4. 확률 분포 엔트로피 최대화를 위한 그래디언트 계산
5. 계산된 그래디언트를 이용한 파라미터 업데이트
6. 2번부터 반복

단점 : 파라미터에 대한 선택이 필수



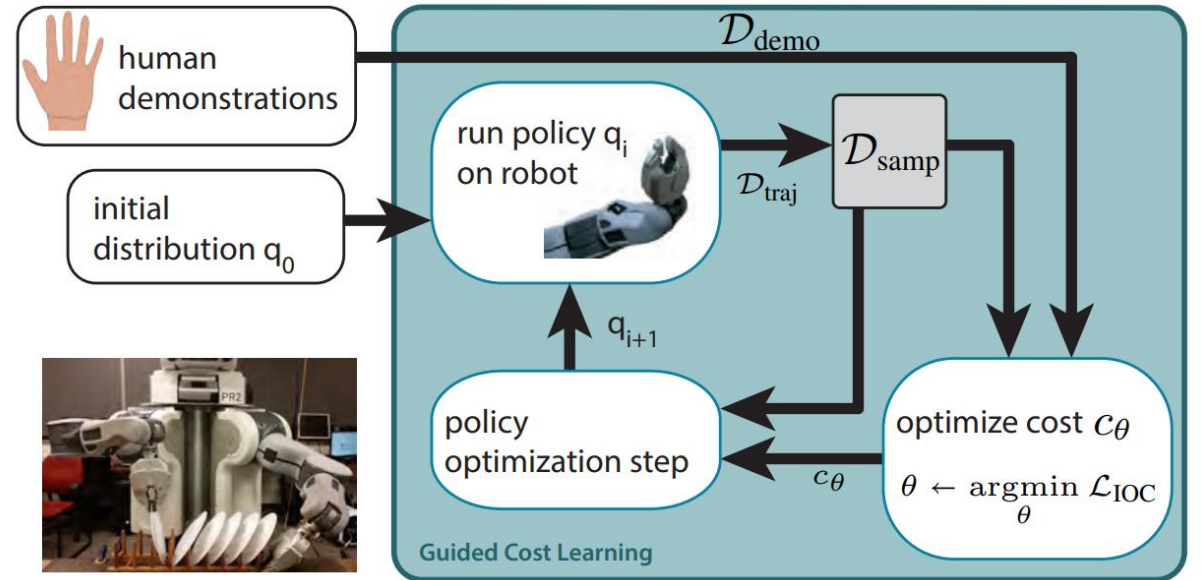
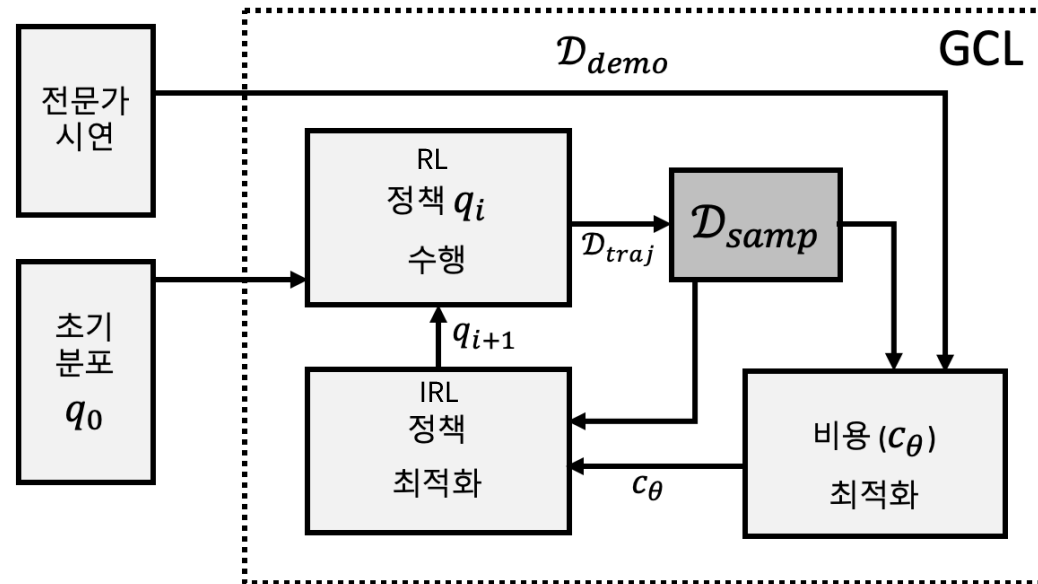
비모수 방식의 **GPIRL(Gaussian Process IRL)**이 도입 -> GP의 특성상 개수에 비례하여 계산 복잡성이 증가하는 단점을 가짐
확률분포가 아닌 복잡하고 비선형적인 심층 신경망 방식인 **DeepIRL**이 도입 -> 복잡도가 샘플 개수와 무관하다는 단점을 가짐

2. Algorithms

3. GCL(Guided Cost Learning)

ALIRL, MaxENT IRL : RL을 이용해 최적의 정책 도출 후 이를 IRL을 이용하여 업데이트(시간, 비용낭비)

GCL : RL로 최적의 정책을 한번에 찾지 않고, IRL과 번갈아 가며 서로 동시에 업데이트 진행



2. Algorithms

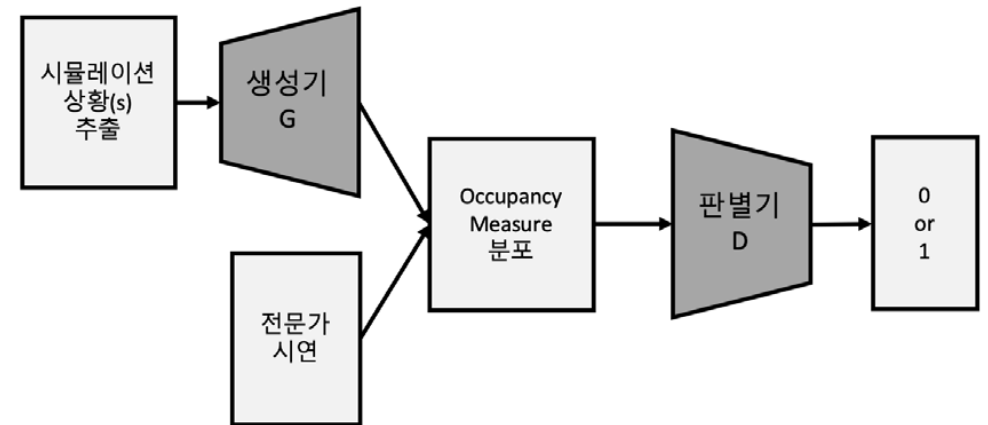
4. GAIL (Generative Adversarial Imitation Learning)

이전까지의 IRL 방법론

1. 정책 초기화
2. 현재 정책과 전문가의 정책을 비교하여 그래디언트 계산
3. 계산된 그래디언트를 이용하여 보상 함수 업데이트
4. 현재 보상 함수로부터 최적의 정책 계산 -> 많은 시간과 자원이 필요함
5. 2)번부터 반복하여 수행

GAIL (Generative Adversarial Imitation Learning)

- **생성기(Generator) = 정책(policy)** : 전문가의 정책을 모방하기 위해 계속해서 유사한 정책 생성
- **판별기(Discriminator) = 보상 함수(Reward)** : 전문가가 수행한 행동인지 아닌지를 구별하는 역할
- **Occupancy Measure** : 특정한 정책 아래에서 발생하는 상태-행동 쌍의 정규화되지 않은 분포를 의미, 해당 분포를 convex conjugate 함수로 변형하여 수식을 단순화함으로써 기존의 GAN과 유사한 목적함수로 변형해주는 역할
- **TRPO(Trust Re-gion Policy Optimization)** : RL 과정에서 정책이 급하게 변하는 것을 방지하는 역할



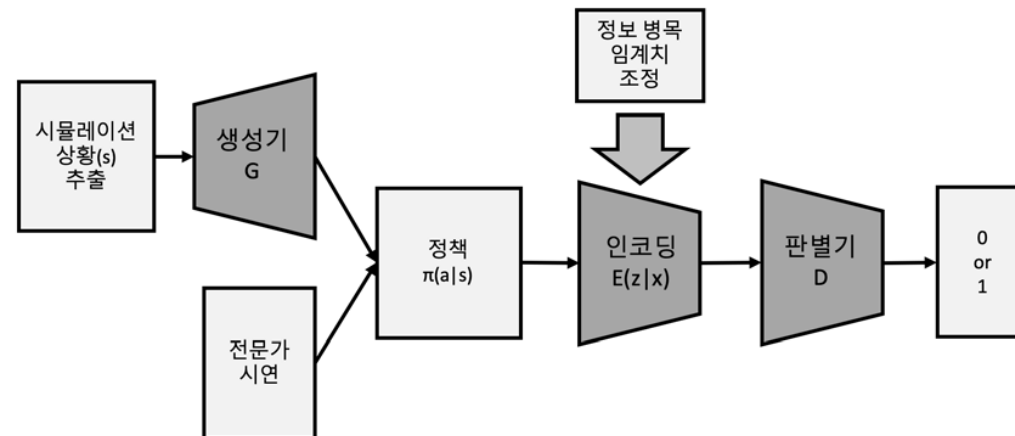
2. Algorithms

5. VAIL(Variational Adversarial Imitating Learning)

GAIL : 생성기와 판별기 사이의 학습 불균형으로 인한 IL 성능 저하 문제 존재

VAIL(Variational Adversarial Imitating Learning)

- 기본적인 구조는 GAIL과 유사함
- 상호 정보량(Mutual Information) : 인코딩을 통해 정보를 제한하는 과정에서 사용(KL-divergence)
- 정보 병목 과정을 통하여 판별기(reward)의 빠른 학습을 억제하고, 생성기의 불완전한 학습을 방지하는 역할을 수행함 또한 불필요한 정보를 무시함



2. Algorithms

6. InfoGAIL: Interpretable Imitation Learning from Visual Demonstrations

GAIL : 전문가 시연 데이터에 존재하지 않는 행동에 대해서는 학습을 잘 구별하지 못함

InfoGAIL: Interpretable Imitation Learning from Visual Demonstrations

- Info - GAN의 방식과 유사하게, Latent Space와 궤적 간의 상호 정보량을 근사적으로 최대화
- InfoGAIL = GAIL + Reward Augmentation
- WGAN(Wasserstein GAN)을 최적화 과정에서 도입 -> 일반적인 GAN의 학습 문제점들 극복

Algorithm 1 InfoGAIL

Input: Initial parameters of policy, discriminator and posterior approximation $\theta_0, \omega_0, \psi_0$; expert trajectories $\tau_E \sim \pi_E$ containing state-action pairs.

Output: Learned policy π_θ

for $i = 0, 1, 2, \dots$ **do**

 Sample a batch of latent codes: $c_i \sim p(c)$

 Sample trajectories: $\tau_i \sim \pi_{\theta_i}(c_i)$, with the latent code fixed during each rollout.

 Sample state-action pairs $\chi_i \sim \tau_i$ and $\chi_E \sim \tau_E$ with same batch size.

 Update ω_i to ω_{i+1} by ascending with gradients

$$\Delta_{\omega_i} = \hat{\mathbb{E}}_{\chi_i} [\nabla_{\omega_i} \log D_{\omega_i}(s, a)] + \hat{\mathbb{E}}_{\chi_E} [\nabla_{\omega_i} \log(1 - D_{\omega_i}(s, a))]$$

 Update ψ_i to ψ_{i+1} by descending with gradients

$$\Delta_{\psi_i} = -\lambda_1 \hat{\mathbb{E}}_{\chi_i} [\nabla_{\psi_i} \log Q_{\psi_i}(c|s, a)]$$

 Take a policy step from θ_i to θ_{i+1} , using the TRPO update rule with the following objective:

$$\hat{\mathbb{E}}_{\chi_i} [\log D_{\omega_{i+1}}(s, a)] - \lambda_1 L_I(\pi_{\theta_i}, Q_{\psi_{i+1}}) - \lambda_2 H(\pi_{\theta_i})$$

end for

3. Conclusion

Conclusion

- 여러가지 다양한 IRL 기법들을 소개함
- 현재 IRL은 강화학습 분야에서도 상대적으로 덜 연구된 분야임.
- 현재 생성형 AI의 여러 기법들(Ex, diffusion model)을 Reward 생성에 도입해보면 어떨까 생각이 들었음.
- 특히 의료 분야에서 reward function을 추정할 수 없는 경우가 많은데, 해당 케이스들에 대해 IRL을 사용한다면 임상적으로 의미 있는 결과를 낼 수 있으리라 생각됨

Next step

- 다음 랩미팅에서는 헬스케어 분야에서의 IRL 적용사례 및 본인 연구에서의 IRL 알고리즘 적용 및 결과를 발표할 예정입니다.

THANK YOU
